

Ağır Rejim, Hafif Rejim

Ömer Faruk Yavuz

August 2026

Özet

Bir yazılım projesinde ne kadar mühendislik titizliği uygulanacağı sorusu, yaygın olarak ürünün türüne göre cevaplanır: girişim ürünleri hızlı ve az testle, kurumsal ya da kritik ürünler ağır süreçlerle geliştirilir. Bu ayrım kaba biçimde işlese de iki belirleyici ölçütü dışarıda bırakır.

Birincisi geri alınabilirliktir. Can kaybı riski taşımayan ve küçük ölçekte çalışan bir üründe dahi, hatası geri alınamayan bileşenler bulunur; veri göçü, para transferi, silme işlemleri ve dışarıya gönderilen mesajlar bu sınıftadır. Bu bileşenlerin taşıdığı risk ürünün genel risk düzeyinden bağımsızdır.

İkincisi zamandır. Titizlik düzeyi bir kez verilip bitirilen bir karar değildir. Ürün ayakta kaldıkça, ölçeği büyümese bile değişiklik maliyeti artar ve başlangıçta rasyonel olan hafiflik bir noktadan sonra ilerlemeyi engeller. Kararın yeniden verilmemesi, sistemlerin “her değişiklik başka bir şeyi bozuyor” durumuna ulaşmasının başlıca nedenidir.

Bu iki ölçüt eklendiğinde karar ürün düzeyinden bileşen düzeyine iner. Aynı ürün içinde iki farklı rejim eşzamanlı olarak çalışabilir: geri alınamaz ve değeri yüksek bileşenlerde ağır rejim, sonucu ucuz düzeltilebilen bileşenlerde hafif rejim.

Girişim yaklaşımının doğru okuması test yazmamak değil, henüz doğrulanmamış bir ürün varsayımına pahalı yatırım yapmayı ertelemektir. Bu okuma, ucuz doğrulama katmanlarının hiçbir aşamada terk edilmemesi gerektiği sonucunu verir; ertelenmesi rasyonel olan yalnızca pahalı katmanlardır.

Giriş

Yazılım geliştirmede uygulanacak titizlik düzeyi bir maliyet-fayda kararıdır. Her doğrulama katmanının bir bedeli vardır: geliştirme süresi, bakım yükü ve değişiklik hızının azalması. Bu bedelin karşılığında elde edilen şey, belirli bir hata sınıfının kullanıma ulaşma olasılığının düşürülmesidir. Karar, bu iki büyüklüğün karşılaştırılmasından çıkar.

Uygulamada bu karar çoğunlukla bir ikilikle ifade edilir. Bir uçta hızın önce geldiği, ürünün doğru olup olmadığının henüz bilinmediği ve ağır sürecin israf sayıldığı yaklaşım; diğer uçta her adımın belgelendiği, sınındığı ve onaylandığı yaklaşım bulunur. Ayrımın ölçütü olarak genellikle iki şey kullanılır: hatanın insan hayatını tehdit edip etmediği ve sistemin ölçeği.

Bu iki ölçüt gerekli ancak yeterli değildir. Notta, ikisine eklenmesi gereken iki ölçüt tanımlanmakta ve bunların eklenmesiyle kararın hangi düzeyde verilmesi gerektiğinin değiştiği gösterilmektedir.

Merkezî terimler burada tanımlanır. Geri alınabilirlik, bir hatanın etkisinin makul bir sürede ve veri kaybı olmaksızın ortadan kaldırılabilmesidir. Ucuz katman, uygulanması geliştirme hızını kayda değer biçimde yavaşlatmayan doğrulama aracıdır; pahalı katman ise sürekli emek gerektiren araçtır. Rejim ise, belirli bir bileşene uygulanan doğrulama katmanları kümesidir.

Notun kapsamı bu kararın verilme ölçütleriyle sınırlıdır. Tek tek doğrulama yöntemlerinin nasıl çalıştığı burada ele alınmamakta, yalnızca maliyet sınıfları bakımından anılmaktadır.

Yaygın Ölçütler ve Yetersizlikleri

Can güvenliği ölçütü

Hatanın insan hayatına mal olup olmaması, titizlik düzeyini belirleyen en güçlü ölçüttür ve bu alanlarda uygulanacak yöntemler standartlarla zorunlu kılınmıştır. Ölçüt kendi kapsamında doğru çalışır.

Yetersizliği, olumsuz cevabın bir serbestlik olarak yorumlanmasındadır. Bir hatanın can kaybına yol açmaması, sonucunun hafif olduğu anlamına gelmez. Yanlış hesaplanmış bir bordro, silinmiş bir müşteri kaydı veya yanlış alıcıya gönderilmiş bir bildirim, insan hayatını tehdit etmemesine rağmen geri alınamaz sonuçlar üretir.

Ölçek ölçütü

Ölçek, bir hatanın kaç kullanıcıyı etkileyeceğini belirlediği için maliyet hesabına doğrudan girer. Milyonlarca kullanıcıya hizmet veren bir sistemde saniyeler süren bir kesinti dahi büyük bir toplam etki üretir.

Yetersizliği ise etkinin genişliği ile derinliğini birbirine karıştırmasıdır. On kullanıcı olan bir sistemde o on kullanıcının verisinin tamamının kaybedilmesi, milyonlarca kullanıcının beş dakika boyunca yavaş hizmet almasından daha ağır bir sonuçtur. Ölçek yalnızca genişliği ölçer.

Birinci Eksik Ölçüt: Geri Alınabilirlik

Bir hatanın maliyetini belirleyen en güçlü tek değişken, hatanın fark edildikten sonra etkisinin ortadan kaldırılıp kaldırılamamasıdır. Bu ölçüt ürünün türünden ve ölçeğinden bağımsız olarak çalışır.

Karşıt iki örnek ayrımı görünür kılar. Bir arayüzde yanlış biçimlendirilmiş bir tablo, fark edildiği anda düzeltilir ve önceki durum tamamen geri gelir; hatanın toplam maliyeti, düzeltmenin süresi ile o süre boyunca oluşan rahatsızlıktan ibarettir. Aynı ürünlerdeki bir veri göçü betiği hatalı çalıştığında ise kayıtlar dönüştürülmüş ya da silinmiş olur; hatanın fark edilmesi durumu değiştirmez, çünkü geri dönülecek durum artık mevcut değildir.

İkinci durumda ürünün ölçeği küçük, can kaybı riski sıfır olabilir. Buna rağmen bileşen, ürünün genelinden bağımsız olarak en ağır muameleyi gerektirir.

Geri alnamaz nitelik taşıyan bileşenler tekrar eden bir liste oluşturur:

- Veri şeması değişiklikleri ve veri göçü işlemleri
- Para transferi, tahsilat ve iade işlemleri
- Silme işlemleri, özellikle kalıcı silme
- Dışarıya gönderilen her şey: elektronik posta, bildirim, mesaj
- Üçüncü taraf sistemlere yapılan yazma çağrıları
- Kimlik doğrulama, yetkilendirme ve şifreleme ile ilgili kod
- Dış dünyaya açılmış ve başkalarının bağımlı olduğu arayüz sözleşmeleri

Listenin ortak özelliği, bu bileşenlerin etkilerinin sistemin denetim alanının dışına çıkmasıdır. Gönderilmiş bir bildirim geri alnamaz, çünkü artık alıcıdadır; yapılmış bir dış çağrı geri alnamaz, çünkü karşı sistemin durumu değişmiştir; dönüştürülmüş bir veri geri alnamaz, çünkü önceki hali saklanmamıştır.

Bu son cümle aynı zamanda ölçütün nasıl gevşetilebileceğini de gösterir. Geri alınabilirlik sabit bir özellik değil, tasarımıyla değiştirilebilen bir niteliktir. Bir silme işleminin kaydı işaretleyip saklaması, bir göç işleminin önceki durumu yedeklemesi, bir bildirim gönderiminin önce sıraya alınıp gecikmeli işlenmesi, ilgili bileşeni geri alnamaz sınıftan geri alınabilir sınıfa taşır. Bu, ağır rejimin uygulanmasından çoğu zaman daha ucuz bir çözümdür.

İkinci Eksik Ölçüt: Zaman

Titizlik düzeyine ilişkin karar, bir kez verilip sabit kalan bir karar olarak ele alınır. Bu ele alış, sistemin kendisinin zaman içinde değiştiğini hesaba katmaz.

Lehman'ın yazılım evrimi üzerine formüle ettiği gözlemlerden ikisi bu noktada belirleyicidir: kullanımdaki bir yazılım ya sürekli değişir ya kullanışsızlaşır, ve bilinçli çaba harcanmadıkça yazılımın karmaşıklığı artar (Lehman, 1980). İkinci önerme, ölçek büyümese ve gereksinimler görece sabit kalsa dahi geçerlidir; karmaşıklık kullanıcı sayısının değil, değişiklik sayısının fonksiyonudur.

Buradan çıkan sonuç şudur. Bir ürünün ilk halinde hafif rejim rasyoneldir, çünkü o aşamada ürünün ne olacağı belirsizdir ve doğrulama yatırımının büyük kısmı atılacak koda yapılacaktır. Aynı ürün iki yıl sonra ayakta kaldığında, kod tabanı büyümüş, bağımlılıklar artmış ve her değişikliğin öngörülemeyen sonuç üretme olasılığı yükselmiştir. Başlangıçtaki karar bu yeni durumda artık rasyonel değildir.

Kararın yeniden verilmemesi tipik bir sonuç üretir: ekip, ilerleme hızının sürekli düştüğü ve her değişikliğin başka bir yeri bozduğu bir duruma ulaşır. Bu durum bir yetkinlik sorunu değil, bir zamanlama sorunudur; başlangıçta doğru olan kararın güncellenmemesinden kaynaklanır.

Kararın gözden geçirilmesini gerektiren eşikler, ürünün kendisinden okunabilir:

- Aynı bileşende tekrar eden hataların ortaya çıkması
- Bir düzeltmenin başka bir yerde yeni hata doğurmasının sıklaşması
- Değişiklik yapmadan önce kodu anlamak için harcanan sürenin, değişikliği yapma süresini aşması
- Kodun belirli bölgelerine dokunmaktan kaçınılmaya başlanması
- Ürünün varsayımlarının doğrulanmış olması, yani atılma ihtimalinin ortadan kalkması

Son madde diğerlerinden farklı bir nitelik taşır. Diğerleri sorunun ortaya çıktığını bildirir; bu madde ise yatırımın artık israf olmayacağını bildirir ve dolayısıyla erken bir sinyaldir.

Kararın Bileşen Düzeyine İndirilmesi

Yukarıdaki iki ölçüt eklendiğinde, kararın ürün düzeyinde verilemeyeceği ortaya çıkar. Aynı ürün, farklı risk profilleri taşıyan bileşenlerden oluşur ve bunlara aynı rejimin uygulanması iki yönlü israf üretir: düşük riskli bileşenlerde gereksiz maliyet, yüksek riskli bileşenlerde karşılanmamış risk.

Ağır rejim

Aşağıdaki niteliklerden herhangi birini taşıyan bileşenlere uygulanır: hatası geri alınamaz, parasal sonuç üretir, veri bütünlüğünü etkiler, güvenlik sınırında yer alır, ya da etkisi sistemin denetim alanının dışına çıkar.

Bu rejimde uygulanan katmanlar: tam test kapsamı, değişmez kontrolleri, sıkı tip disiplini, zorunlu gözden geçirme, değişiklik etki analizi ve dağıtım öncesi ek doğrulama.

Hafif rejim

Hatası ucuz düzeltilebilen, etkisi sistem içinde kalan ve deneysel niteliği yüksek bileşenlere uygulanır: arayüz ekranları, raporlar, biçimlendirme, iç kullanıma yönelik araçlar, henüz doğrulanmamış özellikler.

Bu rejimde hız önceliklidir. Doğrulama yükü, ucuz katmanlarla ve üretim tarafındaki görünürlükle karşılanır.

Bölmenin üstünlüğü

Bileşen düzeyindeki bu ayrımın ürün düzeyindeki ikiliğe üstünlüğü şu gözlemle görülür: her girişim ürününün de bir ödeme akışı, bir kimlik doğrulama katmanı ve bir veri göçü betiği vardır. Bu bileşenler, ürünün genel olarak hangi kategoriye girdiğinden bağımsız olarak hiçbir zaman hafif rejime dahil edilmez. Ürün düzeyindeki bir karar bu ayrımı yapamaz; bileşen düzeyindeki karar yapar.

Girişim Yaklaşımının Doğru Okuması

Girişim ürünlerinin az testle geliştirilmesi genellikle disiplin eksikliği ya da bilinçli bir kalite feragati olarak yorumlanır. Bu yorum, yaklaşımın altında yatan rasyoneli görünmez kılar.

Erken aşamadaki bir üründe asıl belirsizlik teknik değil, ürünün kendisine ilişkindir: yapılan şeyin bir karşılığı olup olmadığı bilinmemektedir. Bu koşulda yazılan kodun büyük bir kısmının atılacağı beklenmelidir. Doğrulama yatırımı da kodun kendisine değil, kodun temsil ettiği varsayıma yapılmış olur. Varsayım yanlışsa yatırım tümüyle kaybedilir.

Dolayısıyla ertelenen şey kalite değil, doğrulanmamış bir varsayıma yapılacak pahalı yatırımdır. Bu ayrım pratik bir sonuç üretir: yatırımın maliyeti sıfıra yakınsa, erteleme gerekçesi de ortadan kalkar.

Ucuz katmanlar hiçbir aşamada terk edilmez

Aşağıdaki araçların ortak özelliği, bir kez kurulduktan sonra geliştirme hızını kayda değer biçimde yavaşlatmamalarıdır:

- Tip sistemi ve geçersiz durumun temsil edilemez kılınması
- Statik analiz ve biçim denetleyicileri
- Veri tabanı düzeyindeki kısıtlar
- Sınırdaki doğrulama, yani dış verinin sisteme girdiği tek noktada denetlenmesi

Bunların maliyeti kurulum anında bir kez ödenir ve sonrasında her değişiklikte otomatik olarak işler. Erken aşamada terk edilmeleri, tasarruf sağladıkları için değil, kalite ile hız arasında kurulan yanlış bir karşılık nedeniyle olur.

Ertelenmesi rasyonel olan pahalı katmanlar

Aşağıdaki araçların ortak özelliği ise sürekli emek gerektirmeleridir: geniş birim test kümesi, uçtan uca testler, mutasyon testi, belgelenmiş süreç ve resmî onay adımları. Bunların her biri yalnızca kurulurken değil, kod her değiştiğinde tekrar maliyet üretir. Ürünün kalıcı olacağı belirsizken bu maliyeti üstlenmemek savunulabilir bir tercihtir.

Hafif Rejimin Asgari Koşulları

Hafif rejim, doğrulamanın tamamen terk edilmesi anlamına gelmez. Terk edilen katmanların yerini alan üç mekanizma bulunur; bunlar olmadan hafif rejim hızlı geliştirme değil, görünmeyen borç birikimi haline gelir.

Hata görünürlüğü

Test yazılmadığında, hataların bulunmadığı değil, bulunup bulunmadığının bilinmediği durum oluşur. Bu belirsizliği ortadan kaldıran şey üretim tarafındaki gözlemlenebilirliktir: yapılandırılmış kayıt tutma, hata takip sistemi, temel metrikler ve uyarılar.

Bu mekanizma olmadan hızlı ilerleme iddiası doğrulanamaz; sistemin doğru çalıştığına dair tek kanıt, kimsenin şikâyet etmemiş olmasıdır.

Hızlı geri alma

Test kümesinin sağladığı güvencenin yerini alan asıl mekanizmadır. Bir değişikliğin sonucu kısa sürede geri alınabiliyorsa, hatanın maliyeti kesinti süresiyle sınırlı kalır ve risk yönetilebilir hale gelir.

Bunun gerektirdikleri: küçük ve sık dağıtımlar, önceki sürüme dönüşün otomatikleştirilmiş olması, ve riskli değişikliklerin kod değişikliği gerektirmeden kapatılabilmesi.

Bu mekanizma önceki bölümdeki geri alınabilirlik ölçütüyle doğrudan bağlantılıdır: hızlı geri alma, ürünün mümkün olduğunca büyük bir kısmını geri alınabilir sınıfta tutma çabasıdır.

Gerçek hatadan test üretimi

Her yeni yetenek için önden test yazılmasa dahi, kullanıma ulaşmış her hata için bir test bırakılır. Bu kural iki şey sağlar. Birincisi, aynı hatanın tekrarı engellenir. İkincisi, test kümesi zamanla geliştiricinin hayal gücüne göre değil sistemin gerçek hata dağılımına göre şekillenir.

İkinci sonuç önemlidir. Önden yazılan testler, yazan kişinin hangi durumların riskli olduğuna dair tahminini yansıtır; gerçek hatadan üretilen testler ise gerçekleşmiş olayları yansıtır. Uzun vadede ikinci kaynak, birim başına daha yüksek koruma değeri üretir.

Sonuç

Titizlik düzeyi kararı, ürünün türüne göre değil bileşenin risk profiline göre verilir. Bu profil dört ölçütle belirlenir: hatanın can güvenliğine etkisi, etkilenecek kullanıcı sayısı, hatanın geri alınabilirliği ve ürünün olgunluk aşaması. Yaygın uygulamada yalnızca ilk ikisi kullanılır ve bu, düşük ölçekli ürünlerdeki geri alınamaz bileşenlerin sistematik olarak eksik korunmasına yol açar.

Geri alınabilirlik ölçütünün en verimli yanı, sabit bir kısıt olmamasıdır. Bir bileşenin geri alınamaz olması çoğu zaman bir tasarım tercihinin sonucudur ve tersine çevrilebilir: silme işleminin kaydı işaretleyerek yapılması, göç işleminden önce yedek alınması, dış çağrıların sıraya alınması. Bu düzenlemeler, ilgili bileşene ağır rejim uygulamaktan genellikle daha ucuzdur. Dolayısıyla karar yalnızca “ne kadar doğrulama yapılacağı” değil, “riskin doğrulamayla mı yoksa tasarımıyla mı karşılanacağı” sorusudur.

Zaman ölçütünün sonucu, kararın periyodik olarak yeniden verilmesi gerektiğidir. Bu gözden geçirmenin tetikleyicileri sistemin kendisinden okunabilir; bunların en erken olanı, ürünün varsayımlarının doğrulanmış olmasıdır. Diğerleri – tekrar eden hatalar, düzeltmelerin yeni hata doğurması, anlama süresinin değiştirme süresini aşması – sorunun zaten oluştuğunu bildiren geç sinyallerdir.

Çerçevenin sınırları belirtilmelidir. Bileşenlerin rejimlere ayrılması, aralarındaki bağımlılıkların zayıf olmasını varsayar; hafif rejimdeki bir bileşen ağır rejimdeki bir bileşenin verisini bozabiliyorsa ayırım fiilen geçersizdir. Bu, rejim ayrımının ancak sınırların net olduğu bir tasarımda uygulanabileceği anlamına gelir. Ayrıca ölçütlerin hiçbiri sayısal değildir; “ucuz düzeltilebilir” ile “geri almamaz” arasındaki sınır uygulamada yargıya bırakılmıştır ve bu yargının sistematik olarak iyimser tarafa kaydığı gözlenmektedir.

Açık kalan soru, bir bileşenin hangi rejime ait olduğunun ekip içinde nasıl kayda geçirileceğidir. Karar örtük kaldığında, bileşene sonradan dokunan kişi o bileşenin taşıdığı riskin farkında olmaz ve rejim fiilen uygulanmaz. Bu, kararın kendisinden çok kararın görünürlüğüne ilişkin bir problemdir.

Kaynaklar

- Feathers, M. (2004). *Working Effectively with Legacy Code*. Prentice Hall.
- Lehman, M. M. (1980). Programs, Life Cycles, and Laws of Software Evolution. *Proceedings of the IEEE*, 68(9).
- Leveson, N. G. (2011). *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press.
- Ries, E. (2011). *The Lean Startup*. Crown Business.